



HASHDIT

HashDit

Smart Contract Security

PancakeSwap Cake Token

Security Audit Report

PancakeSwap

02 Jun 2026 – 03 Jun 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	02 Jun 2026 – 03 Jun 2026
Overall Risk	Green
Project Name	PancakeSwap
Github	https://github.com/pancakeswap/cake-token
Commit	main

Smart Contract List

No.	Contract Name	Verdict	Details
1	CakeToken.sol	Green	-

Notes

[Note 1] Dead code in CakeToken

Contract CakeToken.sol

Description

The CakeToken contract and its BEP20 base contain code that can never execute.

- The internal functions `_burn` and `_burnFrom` are defined in BEP20, but no public or external function exposes them, and they are not called from anywhere else in the contract. As a result there is no reachable path that can ever invoke them, so the token has no working burn capability despite carrying burn logic.
- BEP20 declares `Address` as a type extension for `address`, but none of its functions, `isContract`, `sendValue`, `functionCall`, `functionCallWithValue`, are called anywhere in BEP20 or CakeToken. The library contains low-level ETH-forwarding and arbitrary-call helpers whose presence may give reviewers a misleading impression of the contract's actual capabilities.
- Similarly, the `SafeMath` library defines the `sqrt`, `min`, and `mod` functions, none of which are referenced anywhere in BEP20 or CakeToken.

This does not create a direct security risk, but the unused code increases the contract size and deployment cost, adds maintenance overhead, and can mislead readers and integrators into believing a burn mechanism is available when it is not.

Recommendation No fix likely required, considering the CakeToken is a non-upgradeable token

[Note 2] BEP20.mint(uint256) skips delegate vote tracking, inconsistent with CakeToken.mint

Contract CakeToken.sol

Description

The token exposes two separate mint entry points. `CakeToken.mint(address, uint256)` mints tokens and then calls `_moveDelegates` to keep the governance vote checkpoints in sync with the new balances. The inherited legacy overload `BEP20.mint(uint256)` mints tokens to the caller through `_mint` but does not call `_moveDelegates`, so it never updates the vote checkpoints.

JavaScript

```
/// @notice Creates `_amount` token to `_to`. Must only be called by
the owner (MasterChef).
function mint(address _to, uint256 _amount) public onlyOwner {
    _mint(_to, _amount);
    _moveDelegates(address(0), _delegates[_to], _amount);
}

/**
 * @dev Creates `amount` tokens and assigns them to `msg.sender`,
increasing
 * the total supply.
 *
 * Requirements
 *
 * - `msg.sender` must be the token owner
 */
function mint(uint256 amount) public onlyOwner returns (bool) {
    _mint(_msgSender(), amount);
    return true;
}
```

If tokens are ever created through the BEP20.mint(uint256) overload while the recipient has an active delegation, the minted balance will not be reflected in the recipient's delegated voting power. This leaves the vote accounting inconsistent with actual balances, a latent vote desynchronization that would surface if a future owner mints to itself and self-delegates.

Recommendation

No fix is likely required, considering in the current production setup, the owner is the MasterChef contract, which only invokes CakeToken.mint(address, uint256), so the flawed overload is never exercised and the CakeToken.transferOwnership can never be invoked.

[Note 3] Setting devaddr to the zero address permanently bricks reward emissions

Contract CakeToken.sol & MasterChef.sol

The dev() function lets the current devaddr change the developer address. The function performs no validation on the new value, so the current developer can deliberately or accidentally set devaddr = address(0).

Description

JavaScript

```
// Update dev address by the previous dev.
function dev(address _devaddr) public {
    require(msg.sender == devaddr, "dev: wut?");
}
```

```

        devaddr = _devaddr;
    }

```

Once that happens, every subsequent `cake.mint(devaddr, ...)` call inside `updatePool` reverts, because the `BEP20._mint` implementation in `CakeToken` rejects minting to the zero address. Since `updatePool` is on the critical path for reward accounting, this reverting mint blocks `updatePool` for all pools, halting CAKE emissions protocol wide.

JavaScript

```

function _mint(address account, uint256 amount) internal {
    require(account != address(0), 'BEP20: mint to the zero address');

    _totalSupply = _totalSupply.add(amount);
    _balances[account] = _balances[account].add(amount);
    emit Transfer(address(0), account, amount);
}

```

The condition is also irrecoverable. Fixing it would require calling `dev()` again to set a valid address, but the only authorized caller is now `address(0)`, which can never originate a transaction. The result is a permanent denial of service on emissions, although no funds are directly lost.

Recommendation

No fix is likely required, considering it requires the trusted `devaddr` itself, which is currently a [3/7 Gnosis Safe Multisig](#), to supply the zero address, and the condition has never been triggered in 5+ years of operation.

[Note 4] Summed representative votes may exceed totalSupply

Contract CakeToken.sol

Description

Currently, in `delegate`→`transfer`→`recipient-delegate` paths, summed `getCurrentVotes` can exceed `totalSupply()` without over-minting, as documented in the documentation where transfers do not update checkpoints

Example:

- Alice delegates (100 votes), transfers 100 to Bob (Alice's rep still 100),
- Bob delegates (+100) → 200 votes vs 100 totalSupply.

No over-mint or direct fund loss. If governance/analytics assume $\text{sum}(\text{getCurrentVotes}) \leq \text{totalSupply}()$ or votes follow transfers, tallies may be distorted (e.g. >100% of supply). Sellers may keep votes after selling; recipients add votes without clearing stale checkpoints, with governance integrity and “tokens+votes” expectations at risk.

No fix is likely required, considering the voting mechanism is currently unused.

Recommendation

Consider documenting the behavior and/or non-usage of CakeToken voting mechanisms explicitly.

[Note 5] getPriorVotes is not historical token holdings

Contract CakeToken.sol

Description

Currently, `getPriorVotes(account, block)` returns a representative’s checkpoint vote weight at or before that block, not who held how many tokens (`balanceOf`) at that block, wherein checkpoints are written only on mint, delegate, and `delegateBySig`. Transfer functionalities (`transfer/transferFrom`) do not update or correct them.

Description

Example: At block B, Alice has 10 CAKE and delegates → `getPriorVotes(Alice, B)=10`. After B, a later mint (+5) or transfer can change real holdings while the query at B still returns 10 (`test_getPriorVotes_returnsHistoricalSnapshot`). Reflects the known, documented on-chain checkpoint design—not a broken query.

No over-mint or fund loss. Governance or integrations that treat `getPriorVotes` as a token-holdings snapshot may mis-assign voting weight (e.g. “snapshot at block B” copy but on-chain reads checkpoints).

No fix is likely required, considering the voting mechanism is currently unused.

Recommendation

Consider documenting the behavior and/or non-usage of CakeToken voting mechanisms explicitly.

Findings

No findings were discovered in the CakeToken contract.